

PROJECT DESIGN PHASE-II

Technology Stack (Architecture & Stack)

Date	31 January 2026
Team ID	T-12
Project Name	FinVault – Virtual Stock Trading Platform
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table 1 & table 2

Example: Technical Architecture of FinVault Stock Trading Platform

Reference: <https://github.com/Ganesh200-7/SB-Stocks>



Guidelines:

- Include all the processes (As an application logic / Technology Block)
- Provide infrastructural demarcation (Local / Cloud)
- Indicate external interfaces (third party API's etc.)
- Indicate Data Storage components / services
- Indicate interface to machine learning models (if applicable)

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	How user interacts with application e.g. Web UI, Mobile App, Chatbot etc.	React.js, Vite, Chart.js, HTML5, CSS3, Axios
2.	Application Logic-1	Logic for a process in the application (Routing, order placement, dashboard summary)	Node.js, Express.js
3.	Application Logic-2	Logic for a process in the application (User registration, secure password hashing, JWT creation/validation)	JSON Web Tokens (JWT), bcrypt
4.	Application Logic-3	Logic for a process in the application (Multipart/form-data profile picture uploads)	Multer Middleware
5.	Database	Data Type, Configurations etc. (Local development connection for object modeling)	MongoDB (Local), Mongoose ODM
6.	Cloud Database	Database Service on Cloud (Production hosting of User, Stock, and Transaction data)	MongoDB Atlas
7.	File Storage	File storage requirements (Storing user profile avatars in the backend uploads directory)	Local Filesystem Storage (backend/uploads)
8.	External API-1	Purpose of External API used in the application (Fetching stock prices and historical data for interactive charts)	Finnhub Stock API / Alpha Vantage Stock API
9.	External API-2	Purpose of External API used in the application (Future SMTP integration for registration emails and trade alerts)	NodeMailer / SendGrid API
10.	Machine Learning Model	Purpose of Machine Learning Model (Future AI helper model to forecast price trends and suggest buy/sell actions)	Linear Regression / LSTM (Python, Scikit-Learn)
11.	Infrastructure (Server / Cloud)	Application Deployment on Local System / Cloud. Local Server Configuration: Vite (Port 5173), Express (Port 5001). Cloud Server Configuration: Render (Backend API), Vercel (Frontend Client), MongoDB Atlas (Database Cloud)	Localhost, Vercel, Render, MongoDB Atlas

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	List the open-source frameworks used in the application development.	React.js, Node.js, Express.js, Mongoose, Chart.js
2.	Security Implementations	List all the security / access controls implemented, use of firewalls, hashing, token auth, CORS policies etc.	JWT stateless sessions, bcrypt password hashing, CORS configuration, MongoDB Atlas IP Access Lists & IAM database roles

S.No	Characteristics	Description	Technology
3.	Scalable Architecture	Justify the scalability of architecture (3 – tier, Micro-services). FinVault uses a 3-tier client-server-database architecture. Decoupling the frontend (React SPA) from the backend (Express API) allows separate horizontal/vertical scaling of layers. MongoDB Atlas supports automatic database sharding and replica distribution.	MERN 3-Tier Architecture, MongoDB Atlas Sharding
4.	Availability	Justify the availability of application (e.g. use of load balancers, distributed servers etc.). High database availability is guaranteed using MongoDB Atlas 3-node replica sets. Frontend and backend hosting solutions (like Vercel and Render) deploy across multi-zone CDN edges and incorporate automatic failover and load balancing.	MongoDB Atlas Replica Sets, Edge Delivery (CDNs), Render Load Balancers
5.	Performance	Design consideration for the performance of the application (number of requests per sec, use of Cache, use of CDN's) etc. Client utilizes a fast Virtual DOM (React) and debounced search fields to minimize network calls. The backend database uses indexing on frequently queried columns ('symbol', 'email') for rapid queries. Static resources are served via edge CDN caches.	Database Indexing, Debounced Search, Local Storage, React State Optimization

References:

- <https://c4model.com/>
- <https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/>
- <https://www.ibm.com/cloud/architecture>
- <https://aws.amazon.com/architecture>
- <https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>